Expert Systems With Applications 174 (2021) 114582

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Conditional Wasserstein GAN-based oversampling of tabular data for imbalanced learning

Justin Engelmann *, Stefan Lessmann

School of Business and Economics, Humboldt-Universität zu Berlin, Unter den Linden 6, 10099 Berlin, Germany

ARTICLE INFO

Keywords:
Imbalanced learning
Generative adversarial networks
Credit scoring
Oversampling

ABSTRACT

Class imbalance impedes the predictive performance of classification models. Popular countermeasures include oversampling minority class cases by creating synthetic examples. The paper examines the potential of Generative Adversarial Networks (GANs) for oversampling. A few prior studies have used GANs for this purpose but do not reflect recent methodological advancements for generating tabular data using GANs. The paper proposes an approach based on a conditional Wasserstein GAN that can effectively model tabular datasets with numerical and categorical variables and pays special attention to the down-stream classification task through an auxiliary classifier loss. We focus on a credit scoring context in which binary classifiers predict the default risk of loan applications. Empirical comparisons in this context evidence the competitiveness of GAN-based oversampling compared to several standard oversampling regimes. We also clarify the conditions under which oversampling in general and the proposed GAN-based approach in particular raise predictive performance. In sum, our findings suggest that GAN architectures for tabular data and our extensions deserve a place in data scientists' modelling toolbox.

1. Introduction

Classification models support decision-making in many domains. A common modelling challenge involves imbalanced class distributions. In medical data analysis, a classifier detects patients suffering from a disease and those will typically represent a minority group. Marketing campaigns reach out to a large number of clients out of whom only a few respond. Fraud detection is yet another "needle in the haystack" problem, and these are only a few examples from the large space of applications in which class imbalance impedes the accuracy of a classifier. Addressing class imbalance is relevant as classifiers are increasingly used to make significant decisions.

The paper considers a credit scoring context in which classifiers support loan approval decisions by estimating the probability that a borrower will default. The field of credit scoring is often used to test machine learning (ML) algorithms (Lessmann, Baesens, Seow, & Thomas, 2015) and remedies to the class imbalance problem (Brown & Mues, 2012), which has motivated its selection for this paper. Specifically, we focus on PD (probability of default) modelling in which a binary classifier predicts the likelihood of a borrower or credit applicant to default.

It is common practice to mitigate class imbalance by creating synthetic cases of the minority class (Chawla, Bowyer, Hall, & Kegelmeyer, 2002). This balances class distributions prior to training a classifier and may be achieved by interpolating between neighbouring cases of the minority class. However, distance calculation to determine nearest neighbours and interpolation are problematic in high-dimensional feature spaces and when class boundaries are complex (He & Garcia, 2009). These issues have motivated much research to develop sophisticated strategies for oversampling the minority class. The paper aims at adding to the corresponding literature by examining the suitability of generative adversarial networks (GANs)-based oversampling.

GANs have been proposed to generate complex, high-dimensional data (Goodfellow et al., 2014). As such, they can be used for oversampling to generate synthetic minority class examples. However, most GAN research focuses on unstructured data such as images. Many business classification problems, on the other hand, involve structured tabular data. Such data is also the main source of information in credit scoring. Further, customer-facing applications routinely include both, numerical and categorical variables (Coussement, Lessmann, & Verstraeten, 2017). For example, credit scorecards routinely embody categorical variables to capture demographic characteristics of an applicant

* Corresponding author.
E-mail addresses: justin.engelmann@ed.ac.uk (J. Engelmann), stefan.lessmann@hu-berlin.de (S. Lessmann).

https://doi.org/10.1016/j.eswa.2021.114582
Received 30 September 2020; Received in revised form 14 December 2020; Accepted 5 January 2021
Available online 13 January 2021

and encoding features of the credit product. Approaches for using GANs to model tabular data with categorical variables have emerged only recently and are not reflected in the sparse literature on GAN-based oversampling.

The paper contributes to the imbalanced learning literature by providing a comprehensive analysis of alternative, recently proposed GAN-based architectures for oversampling and empirical evidence of their performance. We develop a GAN that can effectively model tabular data with numerical and categorical variables and pays special attention to the target variable of the downstream task. We also contribute to the credit scoring literature by examining the potential of new deep ML techniques, which might deserve a role in credit analysts' toolbox. For example, generative models such as GAN could help improve the risk management of low default portfolios. Empirical results obtained from multiple credit scoring datasets and classification algorithms suggest that the proposed GAN outperforms several variants of SMOTE and random oversampling. An ablation study further confirms the adequacy of the specific architectural choices in our GAN. However, we also find the baseline approach of not oversampling to perform highly competitive. An implication of this result for credit scoring is that the presence of class skew does not necessarily call for remedial actions. Specifically, we examine the antecedents of the strong performance of the baseline and find that oversampling is suitable for datasets with highly nonlinear feature-to-target relationship and when using advanced ML algorithms for scorecard construction.

2. Background

Methods for mitigating class imbalance split into algorithm- and data-level approaches (Sun, Wong, & Kamel, 2009). The former involve adapting an algorithm to imbalanced problems, for example by modifying the underlying loss function to give minority class errors higher weight. The latter balances class distribution by resampling in the form of undersampling of the majority class, oversampling the minority class, or a combination thereof. Data-level approaches are more flexible and easier to deploy because they support any classification algorithm (Sun et al., 2009).

Unlike undersampling, oversampling exploits all available data but may cause overfitting and introduce noise. Possibly the most popular oversampling approach is the Synthetic Minority Oversampling TEchnique (SMOTE) (Chawla et al., 2002), which generates new minority class samples by linearly interpolating between neighbouring minority cases. Nowadays, many real-world applications involve high-dimensional data with numerical and categorical variables. The information content of different variables is often heterogeneous, with some variables containing more information about the outcome than others. Such settings render distance computations to find neighbouring minority cases problematic. Two neighbouring minority cases selected for interpolation could be conceptually different, which would lead to generating a new synthetic minority case that lies in the majority class space. GANs, on the other hand, are able to learn complex, high-dimensional distributions and might be more effective for oversampling.

3. Related literature

The paper marries concepts from the GAN literature on oversampling and that on generating tabular data. In the following, we review both streams to identify the novelty of the paper.

3.1. GAN-based oversampling

GANs are used extensively for data augmentation in computer vision, whereas only a few studies use GANs for oversampling structured data. Fiore, De Santis, Perla, Zanetti, and Palmieri (2019) train a standard GAN on minority class observations and then use the trained generator for oversampling. They compare GAN to SMOTE and no oversampling on a credit card fraud dataset with only numerical variables using one classification algorithm and obtain mixed results.

Douzas and Bação (2018) propose conditional GAN (cGAN)-based oversampling. They use the standard GAN loss function and assess their approach on real-world and synthetic datasets with numerical features. The study finds cGAN to outperform benchmark oversampling methods but does not clarify some relevant details. First, the discussion of results does not differentiate between real-world and synthetic toy datasets. This is important because the algorithm used for generating the toy datasets creates different classes as Gaussian clusters on the vertices of a hypercube. Receiving Gaussian noise as input, the proposed cGAN might be particularly well-suited for this data generation method and might simply learn to map noise to the vertices of the hypercube. A similar approach by Ren, Liu, and Liu (2019) using a more recent Wasserstein GAN loss and an entropy-weighted label vector has shown encouraging results in benchmarks with no oversampling, a SMOTE variant, and simpler GAN-based methods. However, the study is limited to a single classifier and measures performance by classification accuracy, which is less suitable in imbalanced learning. Son, Jung, Moon, and Hwang (2020) expand on Douzas and Bação (2018) by distinguishing borderline minority cases from ordinary minority cases during cGAN and generating only borderline minority cases when oversampling. Their evaluation shows promising results.

Quintana and Miller (2019) employ the Tabular GAN (TGAN) for oversampling a dataset of subjective thermal comfort. To our best knowledge, this is the only study that uses a GAN architecture specifically designed for generating structured data for oversampling and considers data with both, continuous and categorical variables. However, the study is narrow in that it uses a single dataset, provides no comparisons with baseline oversampling techniques like SMOTE, and uses a limited set of classifiers, which lack state-of-the-art approaches like tree-based ensembles. In the specific setting, TGAN provides mixed results compared to no oversampling.

To summarise, all but one of the surveyed studies consider datasets with only numerical variables. The only exception omits relevant classification algorithms and lacks comparisons to other oversampling techniques. Further, the GAN architectures employed often represent classical approaches and do not reflect advancements from the GAN-based tabular data generation literature (see below). Evidence of the potential of GAN-based oversampling for credit scoring is also unavailable in prior work. Regarding oversampling performance, mixed results have been obtained. Overall, this suggests that additional empirical evidence is beneficial to judge the effectiveness of GANs for oversampling.

3.2. Tabular data generation with GANs

The generation of image (e.g. Karras, Laine, & Aila, 2019) and text (e.g. Press, Bar, Bogin, Berant, & Wolf, 2017) data is the predominant application of GANs. Recently, a few approaches for generating tabular data have emerged, usually with the aim of creating privacy-preserving, synthetic versions of an existing dataset but without consideration to oversampling.

The medGAN (Choi et al., 2018) was developed for a dataset of diagnosis codes associated with patient visits, which leads to a high-dimensional, sparse dataset with only binary columns. Notably, medGAN uses the decoder of a pretrained autoencoder to simplify the task of the generator to deal with this unique data structure. Using a pretrained autoencoder has also been successfully employed for other tabular datasets with more common structures with categorical and numerical attributes, and medGAN-like architectures are often used as a baseline in that literature. Baowaly, Lin, Liu, and Chen (2019) report improved results by replacing the standard GAN loss with a Wasserstein GAN gradient penalty (WGANGP) loss, which highlights that more recently proposed loss functions that are effective for image generation are also effective in the context of tabular data.

A few architectures have been proposed for tabular data with categorical and numerical attributes. TGAN (Xu & Veeramachaneni, 2018) uses LSTM-Cells with learned attention-weights in the generator to create columns one-by-one, which should allow the generator to model the relationships between columns. However, the authors' follow-up CTGAN (Xu, Skoularidou, Cuesta-Infante, & Veeramachaneni, 2019) abandons the LSTM-Cells in favour of ordinary fully-connected layers, as the LSTM-Cells are computationally less efficient and did not perform better. They use one-hot encodings and the softmax activation function with added uniform noise to the output for categorical columns and a Gaussian mixture model-based approach for numerical columns. CTGAN also uses a WGANGP loss.

Mottini, Lheritier, and Acuna-Agost (2018) propose an architecture to generate passenger name record data that is used in the airline industry. Notably, they use crosslayers (Wang, Fu, Fu, & Wang, 2017) in both the generator and the discriminator to explicitly compute feature interactions. Furthermore, they use categorical embeddings in the discriminator to pass soft one-hot encodings forward to the discriminator. This enables them to avoid adding noise like TGAN/CTGAN. They also test Gaussian mixture models for numerical columns and find that this reduces the quality of generated data. As loss function, they use the multivariate extension of the Cramér distance, which has been proposed as an improvement of WGAN (Bellemare et al., 2017).

To sum up, recent work on generating tabular data uses advanced GAN loss functions such as WGANGP and can deal with categorical variables. This contrasts the literature on GAN-based oversampling, in which the standard GAN loss prevails and only one study considers categorical variables. Our goal is to design a GAN architecture for oversampling that incorporates recent methodological advancements for generating realistic tabular data. The literature review mandates an architecture with explicit treatment of categorical variables and an experimental design involving multiple datasets with numerical and categorical variables, diverse classification algorithms, and benchmarking to state-of-the-art oversampling techniques.

4. Methodology

4.1. Generative adversarial networks

GANs are a framework for learning generative models through an adversarial process, which jointly optimises two models (Goodfellow et al., 2014). One model, the generator G, aims at generating samples of synthetic data that are indistinguishable from real data. The other model, the discriminator D, tries to discriminate between real examples from the training set and synthetic examples generated by G. GANs have gained popularity as a method for modelling complex data distributions and have achieved impressive results in computer vision (Karras et al., 2019).

4.1.1. Vanilla GAN

It is common to use neural networks for both G and D, which facilitates the approximation of complex, high-dimensional distributions. G receives a vector of noise drawn from an arbitrary distribution $z \sim p_z$ as input and learns to map this to the data space $\mathscr{X}$. D is trained to classify samples as real or fake, while G is trained to minimise $\log(1 - D(G(z)))$, which corresponds to the following two-player minimax-game:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[\log D(\boldsymbol{x})] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]. \tag{1}$$

Given an optimal discriminator, the generator optimises its objective if p_g, the generator's distribution of samples $G(z)$ with $z \sim p_z$, is equal to the real distribution p_{data}, which corresponds to minimising the Jensen-Shannon Divergence (JSD) (Goodfellow et al., 2014). However, GANs with the standard loss are difficult to train (Goodfellow, 2017). Being a two-player game, convergence to an equilibrium is not guaranteed and the losses do not necessarily correlate with sample quality, which makes it hard to gauge whether the generator is improving during training. Mode collapse, where the generator maps all possible values of z to the sample that the discriminator classifies as real with the highest confidence, is a severe problem in GAN training and is the result of the maximin-game being solved instead of the minimax-game.

4.1.2. Wasserstein GAN and Wasserstein GAN Gradient Penalty

The Wasserstein GAN (WGAN) (Arjovsky, Chintala, & Bottou, 2017) aims to address training challenges by optimising the Wasserstein-1 distance (WD) instead of the JSD. The WD between two distributions can be interpreted as the "cost" of the optimal transport plan to move probability mass of one distribution until it matches the other. Thus, if we have two candidate generative distributions, the WD indicates which of the two is closer to p_{data} even if neither has any overlap with it. This contrasts the JSD used by the original GAN loss, which is infinite for non-overlapping distributions. Thus, WGAN allows for powerful discriminators that provide useful gradients to the generator even when the quality of generated samples is still poor which makes training more stable. Calculating the WD is intractable but it can be approximated by changing the GAN objective to:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[D(\boldsymbol{x})] - \mathbb{E}_{z \sim p_z}[D(G(z))] \tag{2}$$

as long as D is a k-Lipschitz function, which is satisfied by clipping if D's weight lie in a compact space $[-c, c]$, e.g. $c = 0.01$. Thus, WGAN can be implemented by removing the logs from the loss function and clipping D's weight. WGAN with weight-clipping generally works better than the standard GAN and lower losses during training are generally a better indication of sample quality. However, weight-clipping constrains the discriminator's capacity, leads to weights being pushed to the two extreme values of the permitted range $[-c, c]$, and can lead to exploding or vanishing gradients. WGANGP (Gulrajani, Ahmed, Arjovsky, Dumoulin, & Courville, 2017) aims to remedy these problems by replacing weight-clipping with a gradient penalty. Exploiting that a differentiable function is 1-Lipschitz if it has gradients with norm of at most 1, the Lipschitz constraint can be enforced softly by penalising the discriminator with a gradient penalty, which leads to the GAN objective:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[D(\boldsymbol{x})] - \mathbb{E}_{z \sim p_z}[D(G(z))] - \lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}}\left[\left(\| \nabla_{\hat{x}} D(\hat{x}) \|_2 - 1 \right)^2\right] \tag{3}$$

where λ is the penalty coefficient. Gulrajani et al. (2017) recommend using a two-sided penalty that also penalises gradients with norm less than 1. Calculating gradients of D everywhere is intractable and thus the gradients are calculated on linear interpolations $\hat{x} \sim p_{\hat{x}}$ between real and synthetic samples, where $p_{\hat{x}}$ is the sampling distribution of those linear interpolations. The gradient penalty acts as a regulariser and ensures that a discriminator trained to optimality would have a smooth, linear gradient guiding the generator to the data distribution but also constrains the discriminator's capacity. WGANGP has gained popularity in the domains of images, text and tabular data, whereas the standard GAN loss has fallen out of favour.

4.1.3. Conditional GAN and auxiliary classifier GAN

Further modification of the generator's and discriminator's objectives to stabilise GAN training include Conditional GAN (cGAN) (Mirza & Osindero, 2014), where the generator is conditioned to create samples belonging to a specific class by appending the class label y to both the generator and the discriminator input. Thus, the generator estimates the distribution of $p_{X|y}$ and the discriminator learns to estimate $D(X, y) = P(\textit{fake}|X, y)$. Conditioning the generator facilitates sampling output belonging to a specific class, while conditioning the discriminator ensures that the generator cannot simply ignore the class label. Conditioning has been shown empirically to make the training process more stable.

Related approaches augment the generator's objective to add a

reconstruction loss based on how well an auxiliary model can reconstruct part of the generator input from the generated samples, which forces the generator to produce varied output. The auxiliary classifier GAN (AC-GAN) (Odena, Olah, & Shlens, 2017) exemplifies this paradigm. The generator is conditioned on the class label while the discriminator is not. Instead, an auxiliary classifier (AC) is introduced that aims to predict the class label of a given sample $AC(X) = P(y|X)$. The cross-entropy loss of the AC is added to the generator loss to encourage the generator to produce samples that recognisably belong to a specific class. In the original formulation, AC and the discriminator share parameters in the hidden layers, which means that the discriminator cannot be conditioned.

4.2. GANs for tabular data

The terms tabular or structured data refer to a two-dimensional matrix X in which rows represent samples x and columns represent variables. Tabular and unstructured data differ in many ways. For example, text and image data display, for each input variable, a fixed vocabulary of tokens or value range representing colour intensity, respectively. In tabular data, the domain of input variables is more diverse and can be different for each column. Furthermore, the order of input variables in tabular data is typically irrelevant. Considering, for example, three input variables {'Income', 'Age', 'Loan Amount'} that characterise credit applications, their (column) order in a tabular dataset does not affect the task of classifying applications into good and bad risks. Contrast this with a two-dimensional representation of text or image data in which neighbouring columns refer to adjacent words or pixels. Changing the order of columns would greatly affect classification tasks such as predicting the next word in a sentence or whether an image shows a face. In fact, the importance of sequential dependencies in text and image data is the main reason why such data is routinely processed using recurrent or convolutional neural networks, which account for corresponding dependencies between input variables (e.g., columns). The peculiarities of tabular data render transferring methodological advancements from the GAN literature on unstructured data challenging. We detail these challenges and our solutions in the following.

4.2.1. Modelling categorical variables

Categorical variables present a challenge for GANs because the generator and the discriminator need to be differentiable. An intuitive way of generating categorical columns involves one-hot encoding of the category and applying a softmax activation to the generator. However, this approach complicates GAN training as it produces "soft" one-hot encodings (e.g. [0.25, 0.50, 0.25]) that the generator can easily distinguish from "hard" one-hot encodings of the real data samples (e.g. [1, 0, 0]). When generating synthetic data after training, one can sample from the soft one-hot encodings to obtain discrete samples. But sampling is not a differentiable operation, and thus cannot be easily used during training. This can be remedied with straight-through estimation (Bengio, Léonard, & Courville, 2013), which samples from the soft one-hot encodings and passes hard one-hot encoded samples to the discriminator during the forward-pass, but then calculates the gradients w.r.t. the original soft samples during backpropagation. This allows training a GAN with categorical outputs but leads to biased gradients. An alternative approach is to use the Gumbel-softmax function (Jang, Gu, & Poole, 2017), which adds noise from the Gumbel distribution to the logits. For a vector of x representing the unnormalised log probabilities for each of the k categories of a categorical variable, the Gumbel-softmax is applied to each element x_i as follows:

$$\text{Gumbel-softmax}(x_i) = \frac{\exp((x_i + g_i)/\tau)}{\sum_{j=1}^{k} \exp\big((x_j + g_j)/\tau\big)} \text{ for } i = 1, \ldots, k \tag{4}$$

where $g_1, \ldots, g_k$ are drawn i.i.d. from $\text{Gumbel}(0, 1)$ and τ is a temperature hyperparameter that controls the diversity of the output. The resulting output vector emulates sampling from the soft one-hot encodings while being fully-differentiable. Embedding layers are yet another way to train GANs to generate categorical variables (Mottini et al., 2018). For each category, the corresponding one-hot vector is passed through its own embedding layer and the embedded representations are input into the discriminator's hidden layers. Embedding layers offer increased representational power for categorical variables and can improve learning in neural networks. Embedding layers can also be used in conjunction with the Gumbel-Softmax, which ensures that the embedded representations of the generated, soft one-hot-encodings are closer in embedding space to those of the real, hard one-hot-encodings.

4.2.2. Modelling numerical variables

While numerical values are easier to generate, they also require attention in the context of tabular data generation. Values in numerical columns are not necessarily real numbers distributed continuously in a fixed interval. We might have integer values, for example, to capture the number of accounts of a credit applicant. More generally, values could occur in specific increments. For instance, we might have a variable "amount of loyalty points" where all values are a multiple of 10. Constraints on the range of feasible values of a numerical variable can lead to a similar problem as the softmax outputs discussed for categorical data above. The discriminator might learn to reject outputs that are not precisely a value that occurs in the original data. But it is difficult for the generator to output exact float values.

Even if the values of a variable are drawn from a continuous interval, they might display multiple modes. In credit scoring, for example, the loss-given-default is a variable that often displays bi-modal distributions with peaks at zero and one (Leow & Mues, 2012). In such a scenario, the generator should learn to generate these specific modes frequently. But generating exact float values is a challenging task for a neural network. Again, the discriminator might learn rejecting all samples that are close to but not exactly equal to such a frequent mode, as such samples have a high probability to originate from the generator attempting to output a frequent mode.

To address problems associated with multi-modality, we propose adding a small amount of Gaussian noise with zero mean to the numerical columns when training the discriminator. In practice, when we min–max-scale numerical columns to $[0, 1]$, we have found that adding noise with a standard deviation of 0.01 works well, although we did not tune this extensively. Thus, for a vector x representing the d numerical columns of a sample, we add noise to each element x_i such that $x_i^{\text{noisy}} = x_i + z_i$, where $z_1, \ldots, z_d$ are drawn i.i.d. from $\mathcal{N}(0, 0.01)$, and pass the noisy vector on to the discriminator. We add the noise to both real and synthetic samples. Adding it to real samples ensures that the discriminator cannot simply reject values that are not precisely equal to a frequent mode of the real data, while adding it to the synthetic samples ensures that the generator is not learning to emulate the noise but instead has an incentive to output values that are identical to a frequent mode in the real data.

Another challenge concerns the fact that numerical columns might be correlated with realisations of the categorical columns. To make it easier for our generator to capture such patterns, we propose to explicitly condition the numerical output on the categorical output. We

call this approach "self-conditioning" as the generator output for the numerical variables is conditioned on its own output for the categorical columns. As correlations are symmetric, we could also generate numerical columns first and categorical columns second instead. However, when using the Gumbel-softmax activation function, noise is added to the categorical output, which is thus not entirely determined by the generator's hidden state. Without self-conditioning numerical outputs on categorical outputs, the generator has to generate the numerical output without knowing which categories were sampled from the categorical output; when self-conditioning on the categorical output, this information is available.

4.3. Our method: cWGAN-based oversampling

4.3.1. GAN objective

To oversample a dataset, we first train a GAN to estimate the distribution of our data and then use the generator to produce additional samples of the minority class. We use a cGAN structure to estimate the conditional distribution $p_{X|y}$. Conditioning the generator on the minority class label $x_{\text{new}} = G(z, y = y^{\text{minority}})$ allows us to sample the minority class explicitly.

We use the WGANGP loss function due to its theoretical and empirical advantages. To emphasise the class label during training, we further augment our loss function by adding an AC loss. This encourages the generator to generate samples that recognisably belong to the given class instead of merely looking plausible given the label. Compared to AC-GAN (Odena et al., 2017), our architecture involves separate networks for the discriminator and the auxiliary classifier. This allows us to use the AC loss while also conditioning the discriminator, which is not possible with the original AC-GAN setup due to the parameter sharing between D and AC. Thus, we have the following two player game:

$$\begin{aligned}\min_G \max_D & \underbrace{\mathbb{E}_{x\sim p_{\text{data}}}\left[D\left(x\right)\right] - \mathbb{E}_{z\sim p_z}\left[D\left(G\left(z\right)\right)\right]}_{\text{Wasserstein loss}} \\ & \underbrace{-\lambda_{GP}\,\mathbb{E}_{\hat{x}\sim p_{\hat{x}}}\left[\left(\parallel \nabla_{\hat{x}} D(\hat{x})\parallel_2 - 1\right)^2\right]}_{\text{gradient penalty}} \\ & \underbrace{+\lambda_{AC}\,\mathbb{E}_{z\sim p_z}\left[BCE\left(AC\left(G\left(z\right)\right)\right)\right]}_{\text{AC loss}}\end{aligned} \tag{5}$$

where AC is our auxiliary classifier network and BCE is the binary cross entropy between the true class label $y \in \{0,1\}$ and the predicted class probability $y_{pred} \in (0,1)$ defined as $BCE(y, y_{pred}) = -(y\log(y_{pred}) + (1-y)\log(1-y_{pred}))$.

As the magnitude of the Wasserstein loss fluctuates during training, we scale the AC loss by a factor λ_{AC}, which is continuously updated to ten percent of the absolute value of $D(G(z))$ for the current batch $\lambda_{AC} = 0.1|D(G(z))|$ to ensure that minimising the Wasserstein loss is the primary objective of the generator. During backpropagation, the scale factor is treated as a constant. Furthermore, during training, we cap the AC loss at 0.3 so that the generator is not penalised for generating samples where the AC predicts the correct class with high confidence. Thus, the generator is not penalised for generating samples that are clearly recognisable as belonging to the conditioned class. This prevents the generator from overfitting on the AC. The value of 0.3 was chosen as a round number and corresponds to a confidence of about 74% but it is a hyperparameter and could be tuned to a specific dataset.

To see the effect of this approach on the generated data, we plot the output of the AC for training and test sets of the UCI adult dataset as well as synthetic data generated by a trained generator in Fig. 1. This shows that the AC loss leads to the generator generating samples that are clearly recognisable as belonging to the conditioned class. In the case of the harder-to-recognise minority class, this leads to synthetic samples that are much easier to recognise than the real minority samples, but the distributions for the majority class show that the generator makes the samples just recognisable enough to avoid the AC loss being applied.

4.3.2. Modelling tabular data

Our GAN architecture is designed to model tabular data with numerical and categorical columns. We model categorical columns by one-hot encoding them and using the Gumbel-softmax activation function with temperature $\tau = 0.66$. In language modelling, τ is commonly annealed during training to force outputs closer to discrete values. In our setting, a probability distribution over the categories of a categorical column can be appropriate and thus we do not anneal τ. We still choose $\tau < 1$ to make outputs more extreme during the initial stages of training to encourage our generator to associate specific values of the latent noise z with specific categories. For numerical columns, we use min–max-scaling to $[0,1]$ and no activation function for the generator output. We considered using sigmoid and rectified linear unit (ReLU) activations, which might seem more natural choices. However, pre-tests on the UCI adult dataset showed their use to lower performance. Unlike sigmoid and ReLU, which constrain the output of the generator, using no activation function implies that our generator can produce values outside the range of the training data and has to learn to constrain its own output to mostly this range. When sampling from the generator after training is completed, we clip numerical values to $[0,1]$. We also add Gaussian noise to the generator input to address problems outlined in Section 4.2.2.

To better model the relationships between variables, we use crosslayers (Wang et al., 2017) in both the generator and the discriminator. Crosslayers support feature interactions by calculating $x_{n+1} = x_0 x_n^T w_n + b_n + x_n$ where x_n is the output of the n-th crosslayer, x_0 is

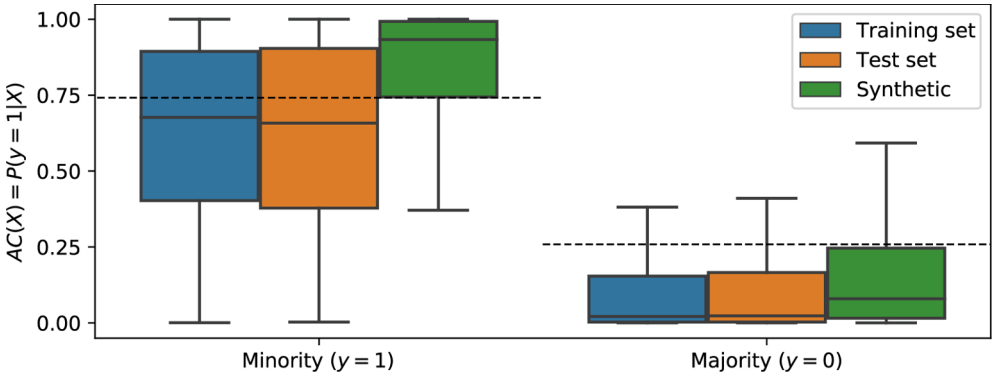

Fig. 1. AC output predicting minority class membership $P(y = 1|X)$ for samples of the UCI adult dataset belonging to the minority class (left) and majority class (right). Dashed lines indicate the cut-off values for the AC loss.

the initial input to the network, and w_n and b_n are the weight and bias of the n-th crosslayer, respectively. Stacking n crosslayers allows for efficient computation of $(n+1)$-th degree feature interactions. In the generator, crosslayers increase the variation of the noise, while in the discriminator and the AC they improve their discriminative power. Furthermore, the crosslayers in parallel to a deep neural network can act similar to a shortcut connection, which are widely used in deep residual networks used for classification (He, Zhang, Ren, & Sun, 2015) and have been used to improve the generator of medGAN (Choi et al., 2018). We also use self-conditioning described in Section 4.2.2 in the generator to further allow the generator to model interactions between variables and to take into account the random sampling of the categorical output by the Gumbel-softmax function.

Furthermore, we use embedding layers for dimensionality reduction and better representative power for the input to the discriminator and the AC, as well as for self-conditioning in the generator. An embedding layer is a single weight matrix that projects sparse one-hot encoded vectors into a lower-dimensional dense vector of real values. For hard one-hot encodings, it acts as a look-up table and in the case of soft one-hot encodings generated by the generator, it outputs a weighted average representation, which alleviates some of the problems outlined in Section 4.2.1. We initialise the embedding layers at random and train them through backpropagation with the rest of the parameters. We set embedding dimensions to $d_{\text{emb}} = min(\lceil k/3 \rceil, 20)$ meaning that a one-hot vector of length k is reduced to a d_{emb}-dimensional dense vector.

We sample our noise from a uniform distribution instead of a normal distribution because we use no activation function for numerical outputs. When sampling from a normal distribution, our noise can occasionally take on extreme values, which then propagate through the generator and cause extreme values for the numerical output. Crosslayers would exacerbate this problem. We also use a noise vector of length 30, which is shorter than most values used in the literature that have been focused on images. Our testing suggests that a short noise vector works better for tabular data. In the future, it might be worth to explore scaling this value with some dataset characteristic, such as the number of clusters identified by a clustering algorithm.

4.3.3. Network structures

The general architectures of our generator and discriminator networks is shown in Fig. 2, 3, respectively. The discriminator receives a sample of data as input with the associated class label y. The categorical columns X_{cat} are embedded by passing the one-hot vector belonging to a categorical column in the original data through the corresponding

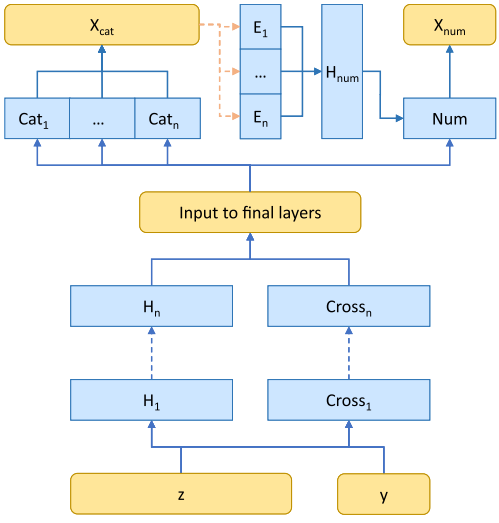

Fig. 2. Structure of the generator.

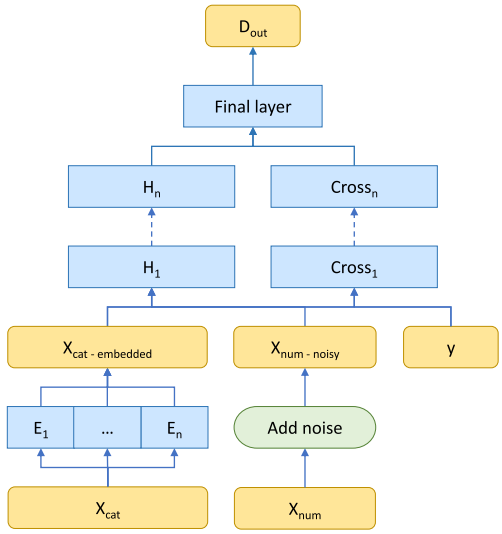

Fig. 3. Structure of the discriminator.

embedding layer $E_1,\ldots,E_n$, and Gaussian noise sampled from $\mathcal{N}(0, 0.01)$ is added to the numerical columns. The embedded categorical columns and the noisy numerical columns together with y are then passed through both the hidden layers $H_1,\ldots,H_n$ and the crosslayers $Cross_1,\ldots,Cross_n$ in parallel. The final layer of the discriminator outputs a scalar, D_{out}, and uses no activation function. We use layer normalisation after each hidden layer in the discriminator.

To generate synthetic data, we first take a sample of latent noise z and a class label y, which is passed through both the hidden layers $H_1,\ldots,H_n$ and the crosslayers $Cross_1,\ldots,Cross_n$ in parallel. The outputs of the final hidden layer and the final crosslayer are then concatenated and form the input to the output layers. We have one output layer for each categorical column $Cat_1,\ldots,Cat_n$, and one for all numerical columns jointly Num. Each categorical output layer Cat_i outputs a vector of length k_i, the number of categories in the i-th categorical column. The numerical output layer outputs a vector of length equal to the number of numerical columns. We concatenate the categorical outputs X_{cat} to the input to the numerical output layer to allow the generator to explicitly condition the numerical output X_{num} on X_{cat}, but we do not propagate gradients back through this connection. To make this more effective, we first embed each categorical column with an embedding layer and then use a further hidden layer H_{num} to reduce the embeddings of all categorical columns to a single 16-dimensional dense vector.

The AC uses the same general structure as the discriminator with three minor modifications. Firstly, no noise is added to the numerical columns; secondly, the class label is not appended to the input; thirdly, a sigmoid activation function is applied to the output to obtain a class probability. The AC is pretrained for 30 epochs and then held constant for the rest of the training. The code of the cWGAN-based oversampling method is publicly available on Github[1].

4.3.4. Generative performance of our method

While oversampling performance is our primary objective, we also want to check whether our cWGAN can successfully learn to generate realistic (synthetic) data. For this purpose, we evaluate the generative performance of the model on the UCI adult dataset that we used for developing our architecture and testing our implementation. No definitive method for measuring the similarity between structured datasets has emerged yet, so we use several different metrics.

First, we visually compare the distributions of each variable indi-

[1] https://github.com/justinengelmann/GANbasedOversampling

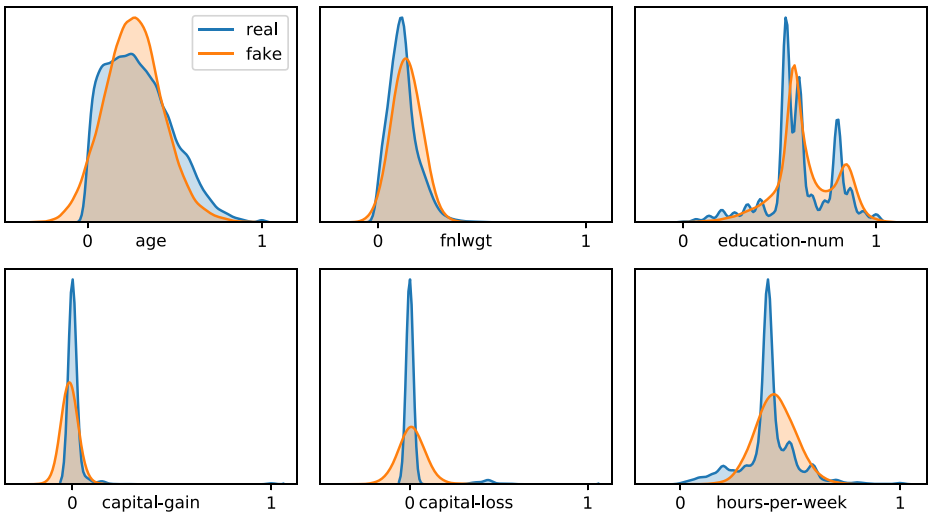

Fig. 4. Univariate distribution plots comparing the real and generated distributions of the numerical columns of the UCI Adult dataset. We plot Gaussian kernel density estimates with a bandwidth of 0.02.

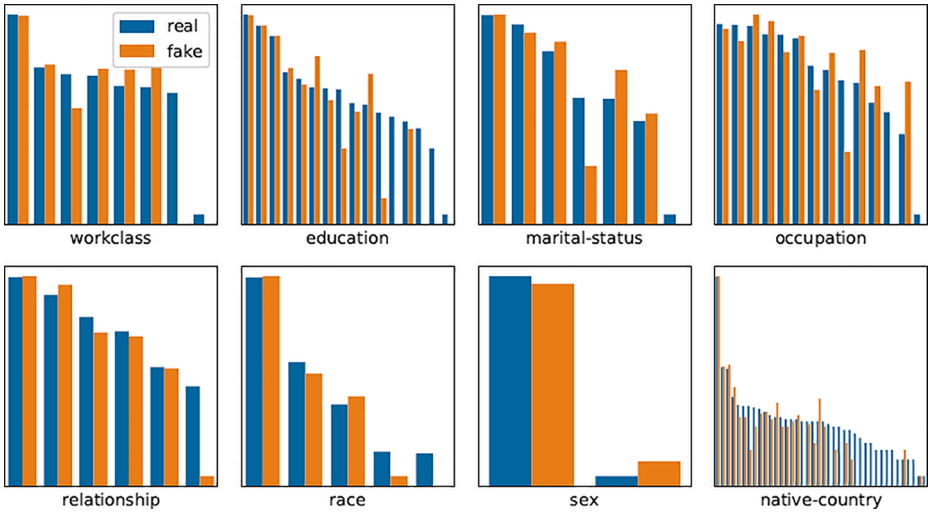

Fig. 5. Univariate count plots comparing the real and generated distributions of the categorical columns of the UCI Adult dataset. Counts are in log-scale and sorted by frequency in the real data to enable easier comparison for columns with categories that occur infrequently.

vidually between synthetic and real data. For numerical variables, we use kernel density estimates with Gaussian kernels and a bandwidth of 0.02 (Fig. 4). For categorical variables, we compare the log-normalized counts for each category (Fig. 5). For numerical columns, our cWGAN generally approximates the distribution of individual variables well and manages to capture multiple modes of a distribution while struggling to differentiate between neighbouring modes. We also find that the generated values can violate implicit constraints on the numerical column in question, such as all values being integers before min–max scaling. If generating realistic data was our primary concern, we could map generated values that violate such a constraint to their nearest legal value, for instance. Such post-processing approaches would increase the complexity of our method and are not necessary for training cWGAN as the adding of Gaussian noise prevents the discriminator from using such violations to identify generated samples. Thus, they are not considered here but could be explored in future work. Some generated values are also outside the data range of the training data. This could be plausible for some features (e.g. height of a person), and implausible for others (e.g. age of a person cannot be negative). Thus, for generating realistic data we might want to allow such deviations for some columns and not for others. For the purpose of oversampling, however, we clip all values to $[0, 1]$ to prevent tree-based classifiers from learning to isolate such values with a single split as they would all belong to synthetic minority class samples. For the categorical variables, we find that cWGAN manages to generate most of the categories present in the real data with approximately the right frequency but fails to generate some of the less frequent categories.

To quantify the generative performance in terms of univariate distributions, we plot a scatterplot of the dimension-wise mean and standard deviation of the synthetic against the real data (Fig. 6, left and middle, respectively). We then quantify the deviation from the identity line with the root mean squared error (RMSE) and also report the Pearson correlation coefficient. We find our cWGAN to match the means and the standard deviations of the original data closely, indicating that the generated data follows the original distribution and does not suffer from mode collapse.

However, these metrics only reflect univariate distributions. To evaluate whether our generator also correctly models the relationships between variables, we also plot dimension-wise prediction performance (Fig. 6, right), a metric from the synthetic data generation literature that

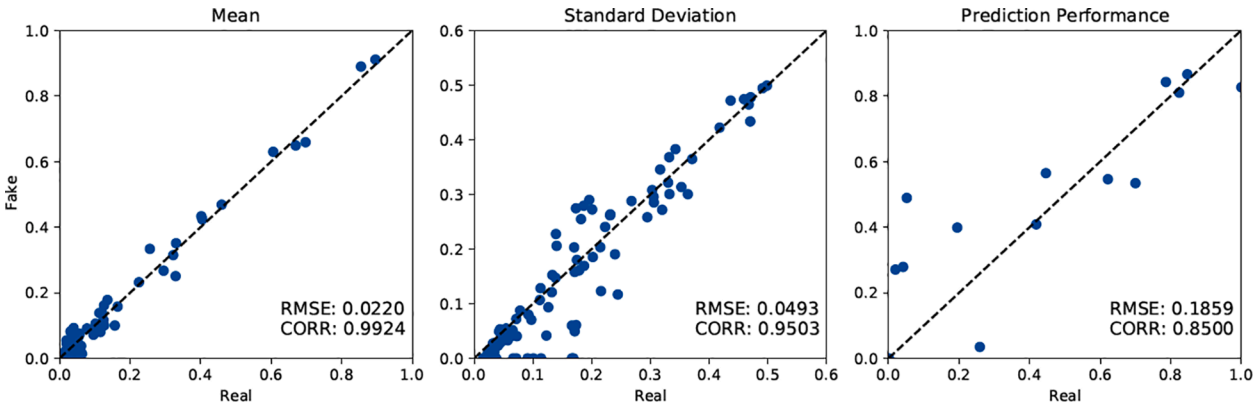

Fig. 6. Dimension-wise performance metrics on the Adult dataset: dimension-wise means (left), standard deviations (middle), and prediction performance (right).

we extend from numerical variable to both numerical and categorical variables. For both the real and the synthetic datasets, we iterate over all columns. The current column is used as the target variable while the remaining columns are used as covariates to train a supervised prediction model. For categorical columns, we use a random forest model and quantify prediction performance with the weighted f_1-score. For numerical columns, we use a Ridge regression and compute the R^2. Models are trained on 90% of the data and scores are calculated on the remaining 10%. This metric captures how predictable a column is given the remaining columns, which is a measure of the strength of association between columns. Ideally, those should be identical between real and synthetic data. As with the previous two metrics, we plot values for the real data against those of the synthetic data and summarise it with the RMSE and Pearson correlation coefficient from the identity line. As it is trivial to predict a column of a one-hot vector given all other columns, we use all one-hot columns belonging to the same categorical variable jointly as a target variable. Thus, there are fewer markers on this scatter plot than on the previous two. Modelling the relationships between variables is a challenging objective for our generator. Early in training, variables are either perfectly predictable or unpredictable but this improves during training.

Overall, we find that our architecture successfully estimates the distribution of a complex, tabular dataset with numerical and categorical attributes and generates synthetic yet realistic data. This is encouraging, especially considering that the AC loss encourages our generator to generate samples that are recognisable as belonging to the conditioned class and thus is biased away from generating modes that occur in the original data but do not clearly belong to a specific class.

5. Experimental design

5.1. Benchmark methods

To assess the proposed cWGAN-based oversampling approach, we consider the baseline solution of training a classifier on the original imbalanced data. We also compare against oversampling techniques including random oversampling, SMOTE, SMOTE-Nominal Continuous (SMOTENC), ADAptive SYNthetic oversampling (ADASYN) (He, Bai, Garcia, & Li, 2008), and Borderline-SMOTE (B-SMOTE) (Han, Wang, & Mao, 2005). We use the package imbalanced-learn (Lemaître, Nogueira, & Aridas, 2017) for the implementations of the benchmark oversampling methods.

Random oversampling balances class distributions by duplicating existing minority class samples. The sampling is done with replacement and continued until achieving a desired class ratio. SMOTE extends random oversampling by generating synthetic new minority class samples x_{new}. These emerge from choosing a random minority case x_i and creating a linear combination with a neighbouring minority case x_j randomly selected from the k-nearest neighbours on a random point along the line connecting both cases $x_{new} = x_i + \in \times (x_j - x_i)$ where $\in \sim \mathscr{U}[0,1]$. The number of neighbouring cases $k_{\text{neighbours}}$ is a hyperparameter. To deal with categorical variables, Chawla et al., 2002 propose SMOTENC, which involves generating the continuous variables of a new minority class sample with SMOTE and then setting the nominal features to the most-frequent value among the k-nearest neighbours.

B-SMOTE concentrates on borderline minority class samples by first finding the m-nearest neighbours of each minority class sample. Samples where the share of majority class samples among the m neighbours is in $[0.5, 1)$ are in danger of being misclassified and used to generate synthetic minority cases with the SMOTE algorithm. Samples below and above this range are considered as safe and noisy, respectively, and are excluded from being selected for generation of synthetic samples. ADASYN works similar to SMOTE but selects minority class samples x_i proportionally to the majority class cases in the k-nearest neighbours of both classes so more cases are created for minority samples that are neighboured by more majority samples.

We test each resampling method together with five classification algorithms, including a random forest classifier (RFC), logistic regression (Logit), a gradient boosting classifier (GBC), k-nearest-neighbours (KNN), and a decision tree classifier (DTC). Due to computational constraints, we use the same set of reasonable default settings for the hyperparameters for all experiments and document our choices in the online appendix[2]. Many other classifiers exist but our selection comprises a diverse set of approaches (regression-based, tree-based, distance-based, ensemble) and includes standard classifiers for credit scoring (Lessmann et al., 2015). Further, our motivation to include DTC and KNN is that they should be particularly sensitive to the quality of the generated minority class cases. DTCs are grown in a greedy manner and vulnerable toward overfitting. As the only synthetic samples in the training data are of the minority class, the trees could fit on spurious patterns in the generated data and, in turn, perform badly on the test set. Good performance of an oversampling technique in conjunction with a DTC, therefore, evidence high quality of the generated minority class cases. KNN is also sensitive to the quality of generated synthetic minority samples. Compared to DTC, KNN yields softer and non-orthogonal decision boundaries that are highly dependent on local training examples. With KNN, the oversampling performance depends on whether generated minority case samples lie close to true minority samples in the test set, which is an implicit measure of how well the distribution of the minority class cases has been approximated by the oversampling method.

[2] Supplementary material.

Table 1
Characteristics of the real-world credit scoring datasets used for our evaluation.

Name	Source	Samples	Num. columns	Cat. columns	Total categories	Imbalance ratio
Germany	UCI MLR	1000	7	13	54	42.9
HomeEquity	Baesens et al.	5960	10	2	10	24.9
Kaggle	Kaggle	150000	10	0	0	7.2
P2P	Lending Club	42506	20	12	117	12.5
PAKDD	PAKDD 2010	50000	8	28	386	35.3
Taiwan	UCI MLR	30000	14	9	77	28.4
Thomas	Thomas et al.	1225	11	3	18	35.8

5.2. Metrics

We evaluate the performance of the classifiers with three metrics: the Area Under the Receiving Operator Curve (AUC-ROC), the Area Under the Precision Recall Curve (AUC-PR), and the Brier score. The metrics are robust toward class imbalance and do not require setting a classification cut-off. The latter ensures that observed results do not depend on the approach taken to decide on the cut-off, which benefits generality. The measures also provide a different view on classifier performance. The AUC-ROC and the AUC-PR are ranking metrics, which capture how well a classifier can discriminate between classes. The AUC-ROC weighs both classes equally while the AUC-PR emphasizes the positive class. The Brier score goes beyond a ranking of cases and captures whether the estimated class probabilities are well-calibrated (Bequé, Coussement, Gayler, & Lessmann, 2017).

5.3. Datasets and data organization

We compare the oversampling methods on seven binary credit scoring datasets with different sample sizes and degrees of class imbalance. All but one dataset contain numerical and categorical variables. Table 1 summarizes the data and from which sources they have been obtained. At the time of writing, all datasets are publicly available and their sources are described in detail in Table D.10 of the online appendix.

For each dataset, we conduct six runs per oversampling technique. We first split the available data randomly into a training set of 90% and a test set of 10% of the data. We opt for a large training set as we expect that all oversampling methods have steep learning curves. Specifically, having more minority class samples enables SMOTE-style methods to generate higher quality synthetic samples due to decreasing the expected average distance between real minority samples. More training data also helps to reduce the extent of overfitting for random oversampling and improves the quality of the generated data for cWGAN-based oversampling.

After partitioning, we apply the oversampling techniques to the training data, generate synthetic minority examples, add these to the training set, and continue until class distributions are even. Next, we train a classification model on the balanced training set, using each of the classification algorithms, and obtain predictions for the test set. We bootstrap the predicted test set 100 times and calculate the performance metrics on each draw. Bootstrapping the test set increases the robustness of our estimates of classifier performance.

We perform each run with a different random seed and use the same six data partitions across oversampling methods. This ensures that test sets (and bootstrap samples) are equally difficult for each method (Demšar, 2006). For categorical columns, we first replace missing values with the most frequent value and then one-hot encode the column. For numerical columns, we use mean imputation and min–max scale to $[0,1]$. SMOTENC is applied after imputation and scaling but before one-hot encoding. All other methods are applied after preprocessing.

5.4. Model selection

An important decision in classifier comparisons concerns the handling of hyperparameters. Hyperparameters exist for individual classifiers and for oversampling methods. The latter are relatively more important in this study, which focuses on oversampling. For this and for computational reasons, we refrain from tuning classification algorithms. Table B.6 in the online appendix documents their hyperparameters, which we set according to literature recommendations.

Especially the cWGAN exhibits many hyperparameters. A fully-comprehensive tuning of all hyperparameters for each dataset might give the proposed approach an advantage over the benchmarks. Therefore, we carry out one round of tuning on an external dataset not part of the comparative study. We use the UCI Adult dataset for this purpose and selected the hyperparameters shown in Table B.2 in the online appendix. Then, for each study dataset of Table 1, we reuse the best hyperparameter settings from the Adult dataset but conduct a small cross-validated gridsearch on the training data to allow for some rough adjustment of generator capacity to the complexity to the focal dataset. Specifically, based on the best AUC-ROC score obtained with a RFC, we select the number of epochs to train from $\{300, 500\}$, the generator's hidden layer sizes from $\{(64), (128, 64)\}$, and whether to use an extra hidden layer before the numerical output. As our method is both new and complex, we expect that fine-tuning for a specific dataset could yield sizeable performance gains.[3] We use the default hyperparameter settings used by imbalanced-learn for the SMOTE variants which is $k_{\text{neighbours}} = 5$ for all variants and $m_{\text{neighbours}} = 10$ for B-SMOTE.

6. Results

6.1. Oversampling benchmark

We compare the 7 oversampling methods in conjunction with 5 different classifiers on 7 datasets using 3 performance metrics. This results in 105 comparisons of the oversampling methods. To summarise the results, we report the mean ranking aggregated over our three metrics of each technique per classifier for each dataset in Table 2, mean rankings per classifier and per metric in Table 4, and summarise the relative performance of our method to the benchmarks in Table 5. For transparency, we report the full, non-aggregated ranks for each metric in Table A.1 and the raw numerical scores per metric in Tables A.2-A.4 in the online appendix. Note that SMOTENC is not defined for datasets without categorical columns and thus no SMOTENC results were obtained for the Kaggle dataset, which contains only numerical columns.

Following Demšar (2006), we use the Friedman test with the Iman-Davenport correction to test the null hypothesis that there is no difference in performance between oversampling methods. The Iman-

[3] Given that we fine-tune hyperparameters for cWGAN but not for the benchmark methods, the reader might be concerned that this could be an unfair comparison. Thus, we provide an ablation of not tuning cWGAN in Table 7 and additional sensitivity analyses in section C of the online appendix.

Table 2
Mean rankings per classifier for each dataset. Sorted by best overall rank per dataset. Best rank per dataset and classifier in bold.

Dataset	Classifier Method	RFC	Logit	GBC	KNN	DTC	Overall
Germany	None	1.7	**1.0**	**1.0**	**2.3**	5.0	**2.2**
	SMOTE	**1.3**	3.0	5.0	5.7	2.3	3.5
	B-SMOTE	3.3	4.3	6.0	3.0	**1.3**	3.6
	Random	5.0	4.3	3.7	6.0	2.3	4.3
	ADASYN	3.7	5.3	4.0	3.7	6.0	4.5
	cWGAN	6.7	5.0	2.7	3.3	6.3	4.8
	SMOTENC	6.3	5.3	5.7	4.0	4.7	5.2
HomeEquity	cWGAN	**1.0**	5.3	2.3	5.7	**1.3**	**3.1**
	SMOTE	3.0	3.7	4.3	2.7	2.0	**3.1**
	None	4.3	**1.0**	**1.0**	7.0	3.7	3.4
	B-SMOTE	4.3	4.7	5.0	3.3	4.0	4.3
	ADASYN	5.7	4.7	5.0	**1.3**	6.0	4.5
	SMOTENC	2.7	5.0	7.0	3.3	5.7	4.7
	Random	6.7	3.7	3.3	4.7	5.3	4.7
Kaggle	cWGAN	1.7	3.7	2.7	**1.3**	**2.7**	**2.4**
	None	**1.0**	4.3	**1.3**	2.7	3.0	2.5
	B-SMOTE	3.3	2.3	4.0	2.7	3.0	3.1
	Random	3.0	**2.0**	3.0	5.7	4.0	3.5
	SMOTE	4.7	3.7	4.3	3.3	3.0	3.8
	ADASYN	5.7	5.0	5.7	5.0	5.0	5.3
P2P	None	2.3	**1.3**	**1.7**	3.7	3.7	**2.5**
	cWGAN	1.7	2.3	2.7	3.3	5.3	3.1
	B-SMOTE	4.3	3.7	3.7	**2.3**	**2.0**	3.2
	Random	**1.3**	2.7	3.0	5.7	4.0	3.3
	SMOTE	4.0	4.7	5.0	4.7	3.7	4.4
	SMOTENC	6.7	5.3	6.7	2.7	3.7	5.0
	ADASYN	5.3	6.0	5.3	5.7	5.7	5.6
PAKDD	None	2.0	**1.0**	**1.3**	**1.0**	5.3	**2.1**
	cWGAN	**1.7**	3.7	2.7	2.0	3.7	2.7
	Random	3.3	2.7	3.3	4.0	**2.3**	3.1
	B-SMOTE	3.7	4.3	3.7	5.7	4.3	4.3
	SMOTE	4.3	5.0	4.7	5.3	3.3	4.5
	SMOTENC	7.0	5.7	6.7	3.0	3.0	5.1
	ADASYN	5.7	4.7	5.7	7.0	6.0	5.8
Taiwan	None	**1.3**	**1.3**	**2.0**	**1.0**	2.3	**1.6**
	cWGAN	2.3	3.7	2.3	2.0	**1.7**	2.4
	Random	2.3	2.0	3.0	4.7	2.7	2.9
	SMOTE	4.0	4.0	3.7	3.3	3.3	3.7
	ADASYN	6.0	4.7	5.3	6.7	6.0	5.7
	SMOTENC	6.0	6.0	5.7	4.3	7.0	5.8
	B-SMOTE	5.7	6.3	6.0	6.0	5.0	5.8
Thomas	None	**1.7**	**2.0**	3.7	**1.3**	**1.0**	**1.9**
	Random	1.7	2.0	2.3	4.3	2.0	2.5
	cWGAN	3.0	4.7	**2.0**	3.7	4.3	3.5
	B-SMOTE	3.7	4.7	3.7	2.0	5.7	3.9
	SMOTE	5.0	2.7	3.3	6.0	4.3	4.3
	SMOTENC	6.7	7.0	6.0	5.3	3.7	5.7
	ADASYN	6.3	5.0	7.0	5.3	7.0	6.1

Davenport correction to Friedman's χ_F^2 is given by $F_F = \frac{(n-1)\chi_F^2}{n(k-1)-\chi_F^2}$, where k and n are the number of classifiers and datasets, respectively. F_F is F-distributed with $k-1$ and $(k-1)(n-1)$ degrees of freedom. We apply the Friedman test to the ranks of the oversampling methods for each combination of classifier and metric separately. For calculating the means, we assign SMOTENC the rank of SMOTE for the Kaggle dataset. The results are shown in Table 3.

We can reject the null hypothesis of equal performance for most classifier-metric combinations except for AUC-ROC and AUC-PR for KNN and DTC. This shows that the Brier score is more sensitive to differences in oversampling performance than the other two metrics, which we attribute to the fact that AUC-ROC and AUC-PR are ranking metrics. The class probability estimates p of KNN and DTC are less granular and when evaluated with a ranking metric, these classifiers might not be well-suited for differentiating between oversampling methods.

Table 3
Results for Friedman test with Iman-Davenport correction. Bold indicates $p < 0.05$.

Classifier	Metric	$F_{(6,36)}$	p
RandomForest	AUC-ROC	4.5476	**0.0016**
	AUC-PR	6.5257	**0.0001**
	Brier	5.0541	**0.0008**
Logit	AUC-ROC	7.8345	**0.0000**
	AUC-PR	6.5838	**0.0001**
	Brier	23.3466	**0.0000**
GradientBoosting	AUC-ROC	31.6552	**0.0000**
	AUC-PR	12.1798	**0.0000**
	Brier	25.2783	**0.0000**
KNN	AUC-ROC	1.4667	0.2173
	AUC-PR	1.4445	0.2251
	Brier	4.6924	**0.0013**
DecisionTree	AUC-ROC	2.2508	0.0603
	AUC-PR	1.7474	0.1382
	Brier	19.4211	**0.0000**

We find the cWGAN-based approach to compare favourably to the state-of-the-art in oversampling. It outperforms the SMOTE variants and random oversampling on five out of seven datasets and ties with SMOTE on a sixth. However, we also find not oversampling to perform highly competitive and achieve the best mean rank on five datasets.

Looking at the mean rankings per classifier and metric across all datasets, a similar picture emerges. Not oversampling performs best for each metric and for each classifier, except DTC, which works best together with SMOTE. Our cWGAN method performed consistently well, except when teamed with logistic regression. Among the benchmark oversampling methods, random oversampling performs best overall. B-SMOTE outperforms SMOTE on some datasets, while ADASYN is consistently outperformed by SMOTE. This indicates that the more recent variations of SMOTE do not necessarily outperform their predecessor.

The strong performance of not oversampling might stem from dataset characteristics. For example, the adverse effect of class imbalance on predictive performance is less severe if classes are, to large extent, linearly separable (López, Fernández, García, Palade, & Herrera, 2013). For the following discussion, we consider a dataset to be strongly non-linear if the AUC-ROC obtained on the original, imbalanced data by a RFC exceeds that of logistic regression by at least 0.1. With this definition, only two datasets, Home Equity and Kaggle, are strongly non-linear. In Table 6, we contrast dataset linearity with the performance of our cWGAN and no oversampling.

On the two strongly non-linear datasets, cWGAN performs best, while not oversampling is the preferred approach on the remaining datasets. This suggests a dependence structure between the complexity of classification, the classification algorithm, and the degree to which a potential imbalance between classes should be addressed. Intuitively, if classes are easily separable, a simple (e.g., linear) classifier is suitable. Our results indicate that such classifier will also cope with class skew and not benefit from oversampling. For example, the proposed cWGAN often gives relatively poor results when paired with logistic regression while substantially raising the performance of complex classifiers such as RFC or GBM (Table 4). Such powerful classifiers are needed when class boundaries are complex and highly non-linear. For such cases, Table 6 suggests that remedying class imbalance by oversampling will likely improve performance. The proposed cWGAN appears especially useful for settings with complex class boundaries and class imbalance. Although more complex and resource-intensive than SMOTE-type alternatives, CWGAN benefits from its ability to estimate complex data distributions and could raise classification performance to a larger extent than simpler oversampling regimes.

Table 4
Mean rankings per metric and per classifier, respectively, aggregated over all datasets. Best rank per metric in bold. Sorted by best overall rank.

	Overall	Classifier					Metric		
Method	Mean Rank	RFC	Logit	GBC	KNN	DTC	AUC-ROC	AUC-PR	Brier
None	**2.3**	**2.0**	**1.7**	**1.7**	**2.7**	3.4	**3.0**	**2.3**	**1.7**
cWGAN	3.1	2.6	4.1	2.5	3.0	3.6	3.8	3.5	2.2
Random	3.5	3.3	2.8	3.1	5.0	3.2	3.2	3.2	4.1
SMOTE	3.9	3.8	3.8	4.3	4.4	**3.1**	3.7	3.9	4.1
B-SMOTE	4.0	4.0	4.3	4.6	3.6	3.6	3.5	4.2	4.4
SMOTENC	5.2	5.9	5.7	6.3	3.8	4.6	5.3	5.3	5.2
ADASYN	5.4	5.5	5.1	5.4	5.0	6.0	5.1	5.3	5.7

Table 5
Summary of the performance of our method relative to benchmark methods.

Dataset	Germany	HomeEquity	Kaggle	P2P	PAKDD	Taiwan	Thomas
cWGAN mean rank	4.8	3.1	2.4	3.1	2.7	2.4	3.5
cWGAN rank of mean ranks	6	1 *	1	2	2	2	3
cWGAN outperforms …							
…all SMOTE variants?	×	× *	✓	✓	✓	✓	✓
…Random Oversampling?	×	✓	✓	✓	✓	✓	×
…Not Oversampling?	×	✓	✓	×	×	×	×
cWGAN performs best?	×	✓ *	✓	×	×	×	×
Not Oversampling performs best?	✓	×	×	✓	✓	✓	✓

* tied with SMOTE.

Table 6
Linearity of datasets and performance of our method relative to no oversampling.

Dataset	HomeEquity	Kaggle	Germany	PAKDD	Taiwan	P2P	Thomas
AUC-ROC RFC w/o Oversampling	0.9733	0.8415	0.7605	0.6203	0.7702	0.7539	0.6265
AUC-ROC Logit w/o Oversampling	0.7738	0.6981	0.7455	0.6181	0.7685	0.7630	0.6528
Difference	0.1995	0.1434	0.0150	0.0022	0.0017	−0.0091	−0.0263
Strongly non-linear?	✓	✓	×	×	×	×	×
Not Oversampling performs best?	×	×	✓	✓	✓	✓	✓
cWGAN outperforms Not Oversampling?	✓	✓	×	×	×	×	×
cWGAN performs best?	✓ *	✓	×	×	×	×	×

* tied with SMOTE.

6.2. Ablation study

Our cWGAN incorporates several elements that are not present in the GAN-based oversampling literature. The most important features are the WGANGP loss, the AC loss, and our treatment of categorical columns. To confirm their merit, we provide ablations in Table 7. We test running our method without the WGANGP loss (using the standard GAN loss instead), without the AC loss, without both, and without special treatment of categorical columns (treating each one-hot encoded column like a numerical column instead). We also test our method without fine-tuning hyperparameters per dataset to check whether fine-tuning has given the GAN-based approach an advantage in the previous comparison. To that end, we train cWGAN with the master-configuration of hyperparameters, which we determined based on the Adult datasets (see Section 5.4). Having found oversampling to be ineffective for linear datasets, we provide ablations for the two nonlinear datasets. Due to space constraints, we count the number of classifier-metric combinations for which the ablation performed worse than the full method, and calculate the counterfactual ranks; that is we calculate the rankings with the cWGAN results replaced by the ablation results. There are a total of 15 classifier-metric combinations. For comparison, our method achieved mean ranks of 3.1 and 2.4 on the Home Equity and Kaggle datasets,

Table 7
Ablation study results. “Home” refers to “Home Equity”. “Cat. col.” refers the ablation where categorical columns are treated like numerical columns in the cWGAN. Kaggle dataset contains no categorical columns.

Ablation	WGANGP		AC		WGANGP & AC		Cat. col.	No fine-tuning	
Dataset	Home	Kaggle	Home	Kaggle	Home	Kaggle	Home	Home	Kaggle
Classifier-metric combinations where ablation performs worse	11/ 15	7/ 15	13/ 15	4/ 15	11/ 15	9/ 15	14/ 15	8/15	4/15
Counterfactual mean rank	4.0	2.3	3.8	2.7	3.9	2.5	3.5	3.1	2.5
Counterfactual rank of mean ranks	3	1	3	2	3	2	3	1	1

respectively, and was the best performing method in terms of mean rank on both, meaning it achieved a rank of mean ranks of 1.

We find that all ablations perform worse in terms of mean rank and rank of mean ranks, except for not using the WGANGP loss on the Kaggle dataset. There, the counterfactual mean rank of 2.3 achieved with the standard GAN loss was marginally better than the mean rank of 2.4 achieved with the WGANGP loss. Most ablations also perform worse on the majority of classifier-metric combinations, with the exceptions being not using the WGANGP loss, and not using an AC loss, which outperformed our method on the Kaggle dataset on 8 and 11 of 15 classifier-metric combinations, respectively. However, when not using the AC loss, the mean rank was worse for the ablation, suggesting that the gains were generally minor and outweighed by the losses on the remaining four classifier-metric combinations. Furthermore, when ablating both the WGANGP and the AC losses together, the ablation performs worse on the majority of classifier-metric combinations, and achieves a worse mean rank. While developing our methodology on the UCI adult dataset, we often observed that without an AC loss, the standard GAN loss could lead to mode collapse. This could explain why ablating either the WGANGP loss or the AC loss individually can improve performance slightly, while ablating both does not. Finally, we find cWGAN performance to remain competitive when not fine-tuning hyperparameters per dataset. On the Kaggle dataset, no fine-tuning even performed better for most classifier-metric combinations but these improvements did not translate into better ranks for the corresponding combinations (e.g., because cWGAN remained the best approach). Whenever performance was worse without fine-tuning, however, this also affected the rankings. As a consequence, the mean rank for Kaggle decreases from 2.4 to 2.5 without fine-tuning. In summary, this ablation suggests that cWGAN performs well even without fine-tuning. We provide additional sensitivity analysis in section C of the online appendix, and arrive at a similar conclusion.

We conclude that each of the major elements of our method is useful and important for the performance as the ablations generally performed worse. However, the ablations also suggest that there is still potential in fine-tuning our method. In particular, which GAN loss to use, whether to use the AC loss, and the AC loss hyperparameters might be important hyperparameters to tune.

7. Conclusion

We proposed cWGAN-based oversampling and provide comparisons to benchmark oversampling techniques as well as the baseline of not oversampling. The empirical analysis involved seven real-world credit scoring datasets with five different classification algorithms using three metrics of classification performance. We find that cWGAN-based oversampling compares favourably to alternative oversampling methods, outperforming random oversampling and SMOTE-style methods on the majority of datasets. Ablations confirm that this result is due to the specific GAN architecture that we propose for oversampling tabular datasets. However, we also find not oversampling to perform better than any resampling method on five out of seven datasets. We attribute this result to characteristics of the employed data. Comparing the classification performance of logistic regression to RFC, we find that the majority of the employed datasets appear to be linearly separable to a large extent. Those datasets that exhibit strong non-linearities benefit from oversampling, whereby the proposed cWGAN raises classification performance the most. We explain this result with the complex structure of those datasets, represented by, for example, intricate feature-response relationships, non-normal feature distributions, and a complex covariance structure. Generating synthetic minority class examples when in the presence of such complexities is challenging for any oversampling method, and we find our GAN-based solution to master these challenges better than traditional alternatives of the SMOTE family.

The main implication of our analysis is that class imbalance does not necessarily call for action. For the field of credit scoring, previous studies have argued that datasets often exhibit mild non-linearities (Baesens et al., 2003), which agrees with our findings. More generally, examining the degree of non-linearity in a dataset by a comparison of a simple classifier (e.g., logistic regression) to a complex approach (e.g., RFC) can provide an indication whether oversampling will be beneficial. We recommend analysts to carry out this easy test when facing an imbalanced dataset. A sizeable advantage of the complex classifiers would identify the classification problem as complex. Then, oversampling should be evaluated. Further, if reserving oversampling for settings with complex data, as opposed to using it by default for imbalanced data, there is reason to believe that a GAN-based approach will deliver the most realistic, synthetic minority class samples and best performance. Thus, the approach proposed here is a valuable addition to the oversampling toolbox and might work especially well in those settings where oversampling is most important. However, future research and empirical evidence from other domains are needed to secure this view. Further, emphasising PD modelling in credit scoring, our study is limited to binary classification. Future work could also consider extensions to multi-class settings such as, for example, rating migration analysis, which is a task that also suffers from class imbalance.

CRediT authorship contribution statement

Justin Engelmann: Conceptualization, Methodology, Software, Visualization, Formal analysis, Writing - original draft, Writing - review & editing. **Stefan Lessmann:** Conceptualization, Formal analysis, Supervision, Project administration, Writing - original draft, Writing - review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary data associated with this article can be found, in the online version, at https://doi.org/10.1016/j.eswa.2021.114582.

References

Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. ArXiv pre-print, arXiv: 1701.07875.

Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J., & Vanthienen, J. (2003). Benchmarking state-of-the-art classification algorithms for credit scoring. *Journal of the Operational Research Society, 54*, 627–635.

Baowaly, M. K., Lin, C.-C., Liu, C.-L., & Chen, K.-T. (2019). Synthesizing electronic health records using improved generative adversarial networks. *Journal of the American Medical Informatics Association, 26*, 228–241.

Bellemare, M.G., Danihelka, I., Dabney, W., Mohamed, S., Lakshminarayanan, B., Hoyer, S., & Munos, R. (2017). The Cramer Distance as a Solution to Biased Wasserstein Gradients. ArXiv pre-print, arXiv:1705.10743.

Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. ArXiv pre-print, arXiv: 1308.3432.

Bequé, A., Coussement, K., Gayler, R., & Lessmann, S. (2017). Approaches for credit scorecard calibration: An empirical analysis. *Knowledge-Based Systems, 134*, 213–227.

Brown, I., & Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications, 39*, 3446–3453.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357.

Choi, E., Biswal, S., Malin, B., Duke, J., Stewart, W.F., & Sun, J. (2018). Generating Multi-label Discrete Patient Records using Generative Adversarial Networks. ArXiv pre-print, arXiv:1703.06490.

Coussement, K., Lessmann, S., & Verstraeten, G. (2017). A comparative analysis of data preparation algorithms for customer churn prediction: A case study in the telecommunication industry. *Decision Support Systems, 95*, 27–36.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research, 7*, 1–30.

Douzas, G., & Bação, F. (2018). Effective data generation for imbalanced learning using Conditional Generative Adversarial Networks. *Expert Systems with Applications, 91*, 464–471.

Fiore, U., De Santis, A., Perla, F., Zanetti, P., & Palmieri, F. (2019). Using generative adversarial networks for improving classification effectiveness in credit card fraud detection. *Information Sciences, 479*, 448–455.

Goodfellow, I. (2017). NIPS 2016 Tutorial: Generative Adversarial Networks. ArXiv pre-print, arXiv:1701.00160.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems, 27*, 2672–2680.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved Training of Wasserstein GANs. ArXiv pre-print, arXiv:1704.00028.

Han, H., Wang, W.-Y., & Mao, B.-H. (2005). Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. *Advances in Intelligent Computing, 17*, 878–887.

He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks* (pp. 1322–1328).

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*, 1263–1284.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep Residual Learning for Image Recognition. ArXiv pre-print, arXiv:1512.03385.

Jang, E., Gu, S., & Poole, B. (2017). Categorical Reparameterization with Gumbel-Softmax. ArXiv pre-print, arXiv:1611.01144.

Karras, T., Laine, S., & Aila, T. (2019). A Style-Based Generator Architecture for Generative Adversarial Networks. ArXiv pre-print, arXiv:1812.04948.

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *The Journal of Machine Learning Research, 18*, 559–563.

Leow, M., & Mues, C. (2012). Predicting loss given default (LGD) for residential mortgage loans: A two-stage model and empirical evidence for UK bank data. *International Journal of Forecasting, 28*, 183–195.

Lessmann, S., Baesens, B., Seow, H.-V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research, 247*, 124–136.

López, V., Fernández, A., García, S., Palade, V., & Herrera, F. (2013). An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics. *Information Sciences, 250*, 113–141.

Mirza, M., & Osindero, S. (2014). Conditional Generative Adversarial Nets. ArXiv pre-print, arXiv:1411.1784.

Mottini, A., Lheritier, A., & Acuna-Agost, R. (2018). Airline Passenger Name Record Generation using Generative Adversarial Networks. ArXiv pre-print, arXiv: 1807.06657.

Odena, A., Olah, C., & Shlens, J. (2017). Conditional Image Synthesis With Auxiliary Classifier GANs. ArXiv pre-print, arXiv:1610.09585.

Press, O., Bar, A., Bogin, B., Berant, J., & Wolf, L. (2017). Language Generation with Recurrent Generative Adversarial Networks without Pre-training. ArXiv pre-print, arXiv:1706.01399.

Quintana, M., & Miller, C. (2019). Towards Class-Balancing Human Comfort Datasets with GANs. In Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation BuildSys 2019 (pp. 391–392).

Ren, J., Liu, Y., & Liu, J. (2019). EWGAN: Entropy-based wasserstein GAN for imbalanced learning. *Proceedings of the AAAI Conference on Artificial Intelligence, 33*, 10011–10012.

Son, M., Jung, S., Moon, J., & Hwang, E. (2020). BCGAN-based over-sampling scheme for imbalanced data. In *In 2020 IEEE International Conference on Big Data and Smart Computing (BigComp)* (pp. 155–160).

Sun, Y., Wong, A. K., & Kamel, M. S. (2009). Classification of imbalanced data: A review. *International Journal of Pattern Recognition and Artificial Intelligence, 23*, 687–719.

Wang, R., Fu, B., Fu, G., & Wang, M. (2017). Deep & Cross Network for Ad Click Predictions. ArXiv pre-print, arXiv:1708.05123.

Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling Tabular data using Conditional GAN. ArXiv pre-print, arXiv:1907.00503.

Xu, L., & Veeramachaneni, K. (2018). Synthesizing Tabular Data using Generative Adversarial Networks. ArXiv pre-print, arXiv:1811.11264.